BASELINE: legacy vs hybrid PQC at the default CPU clock: 1000 runs each

Date: 2026-11-11, 09:09–09:29 UTC. Board: VisionFive 2, CPU governor **ondemand** (default; idles at 375 MHz, max 1.5 GHz). Temperature 45.9 °C at start, 45.7 °C at end. Method: 20 blocks x 50 runs per mode, order alternating every block (legacy→hybrid, then hybrid→legacy, ...), every single measurement recorded. 4000 connections, 0 failures. Every block was checked for the negotiated group (X25519 vs X25519MLKEM768) and, on our server, for successful ML-DSA-44 verification. Raw data: `baseline_raw_1000.csv`, per-block log: `baseline_bench_1000.log`, statistics: `baseline_stats_1000.txt`.

Scenario 1: hackathon server over IPv4 (193.30.40.242), TLS handshake time

Legacy = X25519, hybrid = X25519MLKEM768. Server authentication is RSA-3072 in both (the only thing it supports).

Mode	n	Best	Median	Mean	95th pct	99th pct	Worst
Legacy (X25519)	1000	31.61 ms	**39.92 ms**	45.07 ms	70.84 ms	105.36 ms	524.45 ms
Hybrid (X25519MLKEM768)	1000	36.33 ms	**45.08 ms**	50.09 ms	76.03 ms	104.35 ms	359.32 ms

Hybrid − legacy (median): **+5.16 ms (+12.9 %)**. Per block: +5.0 ms median, range −0.9 to +10.6 ms; hybrid was faster in only 1 of 20 blocks.

Scenario 2: own server on the Mac (LAN), end-to-end time

Legacy = X25519 + RSA-3072. Hybrid = X25519MLKEM768 + ECDSA P-256 + ML-DSA-44 (binary signature, fast-P256 OpenSSL), i.e. the best configuration from 2026-11-10.

Mode	n	Best	Median	Mean	95th pct	99th pct	Worst
Legacy (X25519 + RSA)	1000	14.77 ms	**21.61 ms**	21.53 ms	24.35 ms	26.91 ms	31.30 ms
Hybrid (ML-KEM + ECDSA + ML-DSA-44)	1000	20.51 ms	**27.95 ms**	27.99 ms	31.58 ms	33.84 ms	40.05 ms

Hybrid − legacy (median): **+6.35 ms (+29.4 %)**. Per block: +6.2 ms median, range +1.5 to +9.0 ms; hybrid was faster in 0 of 20 blocks.

Comparison with full CPU speed (performance governor, 2026-11-10)

Scenario	Governor	Legacy median	Hybrid median	Difference
Own server, end-to-end	performance (50 runs x 2 passes)	10.66 ms	10.32 ms	−0.34 ms (no increase)
Own server, end-to-end	**ondemand (1000 runs)**	21.61 ms	27.95 ms	**+6.35 ms (+29 %)**
Hackathon server, handshake (IPv6, 2026-11-10)	performance (50 runs x 2)	41.7 / 44.0 ms	45.3 / 45.0 ms	+1–4 ms
Hackathon server, handshake (IPv6, 2026-11-10)	ondemand (50 runs x 2)	57.3 / 56.9 ms	62.2 / 61.1 ms	+4–5 ms
Hackathon server, handshake (IPv4, today)	**ondemand (1000 runs)**	39.92 ms	45.08 ms	**+5.16 ms (+13 %)**

Findings

1. **At the default clock, the "no increase in latency" target is not met.** Hybrid costs +5.2 ms (+13 %) against the hackathon server and +6.4 ms (+29 %) on the LAN. The result is consistent: hybrid lost in 19 of 20 and 20 of 20 blocks.
2. **The target is met only with the CPU at full speed.** With `performance`, hybrid matched the baseline (10.32 vs 10.66 ms). The hybrid result therefore depends on the clock policy, not only on the algorithms.
3. **Why the clock matters more for hybrid:** the legacy client does very little crypto (RSA verify 0.65 ms at 1.5 GHz), while the hybrid client verifies ECDSA (1.4 ms) and ML-DSA-44 (1.0 ms) and runs ML-KEM (~1.5 ms). Under `ondemand` the CPU starts each short burst at 375 MHz (4x slower), so the larger client-side workload is stretched the most. Both modes got slower (legacy 10.7 → 21.6 ms, hybrid 10.3 → 28.0 ms), hybrid by more.
4. **Worst cases come from the network, not crypto.** Against the hackathon server, single spikes reached 524 ms (legacy) and 359 ms (hybrid); the 99th percentiles are almost identical (105 vs 104 ms). On the LAN the spread is small (worst 31 vs 40 ms, standard deviation ~2.3 ms in both).
5. **Thermal trade-off:** at `performance` the board idled at ~50–52 °C; at `ondemand` ~46 °C (both well below critical). This is a real deployment trade-off: low latency for hybrid PQC needs a higher sustained clock, which costs heat and energy.

Possible middle ground (not yet tested)

- Tune `ondemand` to react faster (lower `sampling_rate`, lower `up_threshold`), or use `schedutil`, so the CPU reaches full speed within a handshake without staying there when idle.
- Raise the clock only around connection setup (application-level frequency boost).
- Fix a lower but constant clock (e.g. 1.25 GHz with `userspace`) to trade some latency for less heat.

Source: baseline_REPORT.md